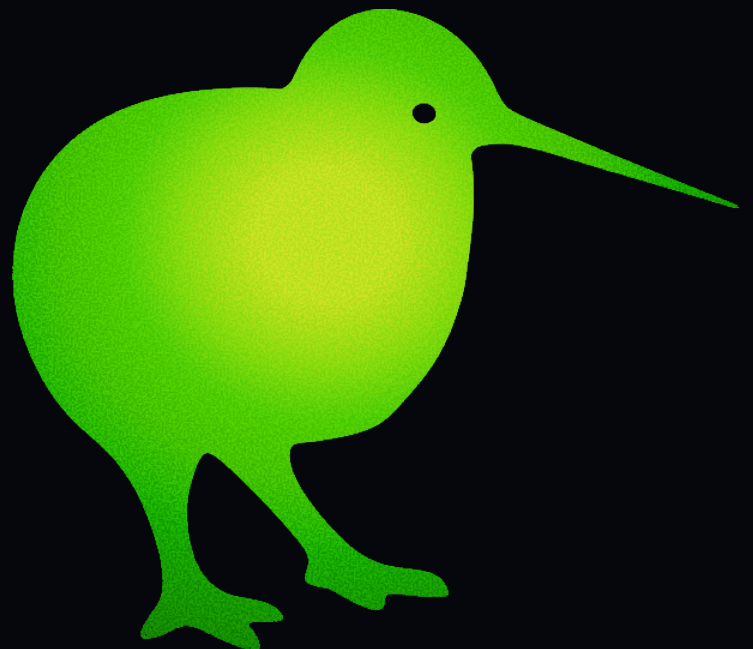


—
ROLE: GOLDEN GOOSE

The Things Kivi Comes to Know

kivi
by Sarvam



Kivi is a voice-first interface for computing. Today, a person can use it in two ways.

In regular dictation, they speak and Kivi writes. Styles determine how that speech should become written language, while phonetic memory helps Kivi recognise the words that belong to the person using it.

In **Hey Kivi**, the person addresses Kivi directly. This is currently a beta feature. We want Kivi to understand a request and use tools to help complete it.

Our long-term direction is for Hey Kivi to become the interface through which a person uses the entire product. Dictation would not disappear; it would become one of the tools available inside that interface.

Styles and phonetic memory account for almost everything required to make ordinary dictation personal and correct.

Semantic memory has a different opportunity. It can allow Hey Kivi to understand what has happened across a person's work and use that understanding when the person asks for help.

Semantic memory begins where a dictionary ends. It is the durable understanding that allows a product to recognise the preferences, facts, and context that continue to matter over multiple interactions.

Your assignment is to decide what semantic memory should mean for Kivi, determine what Hey Kivi should be able to do with it, and build the product and backend end to end.

There are two parts to this assignment. Complete them in order.

PART ONE

Product Position and Vision

Before designing the interface or choosing an architecture, decide what Kivi should know about a person—and why.

Use Kivi. Study how other products approach memory, personalisation, context, and user control. Look beyond feature lists. Consider what kind of relationship each product appears to want with its user and what that relationship permits it to remember.

Then take a position.

Tell us:

- what semantic memory should allow Kivi to become;
- where the real value is created for the user;
- what deserves to be remembered;
- what Kivi must never assume;
- why someone should trust this system enough to keep using it.

Your position should determine the product you build. It should make some technically

impressive ideas inappropriate.

SUBMIT

- a product positioning statement of no more than 100 words;
- a product vision document of no more than 600 words.

This part must be entirely your own thinking. Do not use generative AI to arrive at the position or to write either document for you.

We are interested in how you see: what you notice, what you connect, what you reject, and the conclusion you are prepared to defend. We will examine this reasoning closely during the interview. A borrowed position rarely survives that conversation.

Complete and preserve both documents before beginning Part Two.

PART TWO

Build and Prove It

Design and build one working end-to-end demonstration of the product you described.

There is no separate prototype. The implemented product is the design.

Its interface, interactions, language, visual character, and backend will be evaluated together.

■ Begin with the use case

Work backwards from something genuinely useful.

A memory system may be capable of learning ten things while only three of them create meaningful value for the user. Do not build around everything the system could infer.

Decide which capabilities deserve to exist, then build the product and infrastructure required to make those capabilities trustworthy.

For example, a person might say:

“Hey Kivi, find the dictation I did around 5 PM yesterday in Slack and polish it for the meeting I’m walking into.”

Semantic memory may make factual, episodic, and preference-level understanding possible. Determine which forms of understanding matter for the product you are building, what Hey Kivi should be able to do with them, and which tools those use cases require.

Build the smallest set of capabilities that produces a product worth using.

■ Build the complete experience

The end-to-end demonstration should make clear:

- what the person experiences during ordinary use;

- how Kivi's understanding changes its future behaviour;
- whether that behaviour belongs in regular dictation or Hey Kivi;
- how and when Kivi interfaces with the person when its understanding is incomplete or wrong;
- how the person remains in control without becoming the administrator of the system.

The distinction between regular dictation and Hey Kivi is important. Semantic memory may have little reason to affect ordinary dictation while being central to an interactive request.

Decide what belongs in each mode and make that boundary coherent in the product.

You have complete freedom over how memory appears within the app. You may create a new surface, transform an existing part of Kivi, reconsider the navigation, remove current concepts, introduce new ones, or change the visual identity entirely.

The product should begin and end in an interface intended for a normal user. It should not require a developer console or an explanation of the underlying system to become intelligible.

You do not need to build speech recognition or integrate with production Kivi. You may replay transcripts, application context, and other events through a client of your own design.

■ Build the system beneath it

The visible experience must be connected to a real backend. Its behaviour should come from actual state, persistence, retrieval, and model decisions—not from a sequence prepared only for the demonstration.

Choose the architecture yourself. We will not provide Kivi's memory design, storage model, retrieval method, prompting strategy, or definition of a memory.

Those decisions should follow from the product position and use cases you established.

Your system should determine:

- what it learns and what it deliberately ignores;
- how a memory is created, represented, stored, changed, and removed;
- how factual, episodic, and preference-level understanding relate to the product you chose to build;
- how relevant understanding is retrieved when the person uses Hey Kivi;
- how memory influences tool use, responses, or other visible behaviour;
- how an engineer can inspect why memory did or did not affect a result.

Implement only the Hey Kivi tools required by your chosen use cases. A narrow set of tools used convincingly is more valuable than a broad collection of superficial capabilities.

The system should be narrow enough to finish and complete enough to interrogate. A web or desktop implementation is acceptable. A notebook, static demonstration, prompt collection, or architectural proposal is not.

■ Prove it yourself

You will not receive our user data while completing the assignment.

Create or obtain approximately 500 transcript-like records for development. Each record should

contain raw ASR output, LLM-formatted output, and whatever additional metadata your chosen product genuinely requires.

You may generate this corpus with language models, use suitable open-source data, or combine both.

We are not treating this as a data-generation competition. The corpus only needs enough variety to exercise the system you chose to build and reveal where it succeeds, abstains, or fails.

Build a reproducible evaluation that runs the complete pipeline on this corpus.

For every result, it should be possible to inspect:

- the original input;
- the memory the system created, retrieved, changed, or rejected;
- the provenance of that memory;
- the resulting Hey Kivi behaviour;
- the reason the system made its decision.

Include latency, database growth, model usage, and cost wherever they matter.

Evaluations

There are two stages to our evaluation.

■ Your evaluation

We will first run the evaluation included in your repository.

We will inspect whether it tests the product you actually built, whether the cases are sufficiently varied, whether failures remain visible, and whether the conclusions follow from the evidence.

■ Our evaluation

We will then run your complete pipeline on a separate internal corpus of approximately 500 dictations from one user.

Each record will contain:

- raw ASR output;
- LLM-formatted output;
- the ordinary metadata available in our logs.

Your system must provide a documented way for us to import the corpus, process it, inspect the resulting database and memory state, and operate Hey Kivi against what it has learned.

After ingestion, we will ask Hey Kivi questions grounded in that user's history.

We will examine:

- which facts, preferences, and episodes Kivi learned;

- what it deliberately ignored;
- whether it can recover information distributed across multiple dictations;
- whether it can use that understanding through the Hey Kivi capabilities you implemented;
- whether its answers are supported by the original interactions;
- whether it refuses to invent an answer when the history does not contain one;
- which memories and source interactions produced each answer;
- retrieval latency, end-to-end latency, database growth, model usage, and cost.

“Kivi should answer any question” means any reasonable question whose answer is present in or derivable from the supplied user history.

We will evaluate the interface and backend together. A compelling interface cannot rescue memory that is careless with the truth. A strong backend cannot rescue a product that has no worthwhile use for what it knows.

Submission

Place the complete submission in one GitHub repository.

The repository must contain:

- the product positioning statement;
- the product vision document;
- the complete source code;
- the working interface and backend;
- the database schema and migrations;
- reproducible seed data;
- the corpus and evaluation you created;
- generated evaluation results;
- a README explaining the product, architecture, use cases, limitations, results, and AI use;
- a `RUN.md` declaring the primary review method and explaining exactly how to use it.

Your application may be submitted as:

- a fully hosted application;
- a completely local application;
- a local interface connected to a hosted backend;
- a local interface and backend connected to a hosted database;
- a containerised application and database;
- another hybrid arrangement that the reviewing agent can reproduce.

Your database may be:

- SQLite, DuckDB, or another embedded database;
- Postgres, MySQL, or another locally running database;

- a database started through Docker;
- Supabase, Neon, Railway, or another managed database service;
- part of a fully hosted application.

All of these arrangements are valid. Deployment is not required.

Choose one arrangement as the primary review method. State it at the beginning of `RUN.md`.

For a hosted application, `RUN.md` must provide the direct URL, any required demonstration credentials, the interactions to try, the candidate evaluation procedure, and the corpus-import procedure.

For a local or hybrid application, `RUN.md` must provide:

1. the required runtimes and versions;
2. every required environment variable;
3. the exact commands to install dependencies;
4. the exact commands to create, migrate, and seed the database;
5. the exact commands to start every required process;
6. the URL, application window, or interface to open;
7. the primary interactions to try;
8. the exact command to run the candidate evaluation;
9. the exact procedure for importing another corpus;
10. where evaluation results and memory state can be inspected;
11. the exact procedure for resetting the system.

If an LLM key is required, name the environment variable precisely and include an `.env.example`. Do not commit private credentials.

SUBMIT

Through the application form, provide:

- the GitHub repository URL;
- the exact final commit SHA;
- the hosted application URL, if the primary review method is hosted.

A coding agent will clone the submitted commit and follow the primary review method declared in `RUN.md`.

The agent may install declared dependencies, provide documented model credentials, and translate our internal corpus into the documented import format. It will not infer missing setup, perform undocumented dashboard work, repair the application, or contact you for clarification.

Test the complete review path from the submitted commit before applying.

If the agent cannot seamlessly start, operate, reset, inspect, and evaluate the system, we may be unable to review the submission.